Projektdokumentation

Radio2Xray - C# Konsolenprogramm zur Berechnung der Wellenlänge anhand einer Frequenz

Modul	M319 Applikationen entwerfen und implementieren
Eingereicht von	Jonas Vetsch
Projektthema	Radio2Xray - Berechnung Wellenlänge mit C#
Eingereicht bei	Michael Abplanalp
Datum	20. Oktober 2025

Inhaltsverzeichnis

1 Die Funktion des Programms Radio2Xray 3
1.1 Grundinformationen 3
1.2 Woher kommt die Idee? 3
1.3 Die wichtigsten Bauelemente von Radio2Xray 4
2 Typische Programmdurchläufe 6
2.1 Nutzer macht eine gültige Eingabe 6
2.2 Nutzer macht eine ungültige Eingabe 8
3 Test und Code-Review 9
4 Reflexion 10
4.1 Zielsetzung und Planung 10
4.2 Umsetzung 10
4.3 Schwierigkeiten und Lösungen 10
4.4 Lernprozess 10
4.5 Selbstreflexion und nächste Schritte 10

1 Die Funktion des Programms Radio2Xray

1.1 Grundinformationen

Das Programm Radio2Xray berechnet anhand einer vom Benutzer eingegebenen Frequenz die dazugehörige Wellenlänge.

Dabei wird die eingegebene Frequenz im Hintergrund zuerst in Hertz (SI-Einheit) umgerechnet. Danach wird mit der physikalischen Formel

$$\lambda = \frac{c}{f}$$

die Wellenlänge berechnet, wobei c = 299'792'458 m / s die Lichtgeschwindigkeit im Vakuum ist.

Das Ergebnis (in Meter) kann sehr gross (mehrere Kilometer) oder auch sehr klein (im Mikrometerbreich) sein. Deshalb wird es im Hintergrund zuerst in eine sinnvolle Masseinheit umgerechnet, bevor es dann dem Nutzer ausgegeben wird.

Alle gemachten Berechnungen speichert das Programm zudem in ein Text-File («Berechnungen.txt»), damit der Nutzer auch nach Beenden des Programms seine Berechnungen noch nachvollziehen kann.

Das Programm prüft selbstverständlich die Benutzereingaben auf Gültigkeit (Zahl, positive Werte, Länge der Eingabe), bevor es die Berechnung der Wellenlänge ausführt.

1.2 Woher kommt die Idee?

Die Idee des Programms entstand, weil ich Wellenlängen besser verstehen wollte und in diesem Programm die Gelegenheit sah, ein physikalisches Thema mit Programmierlogik zu verbinden und Einheitenumrechnungen zu automatisieren. Ich denke zudem auch, dass ich dieses Programm in Zukunft selbst verwenden werde, wenn ich Wellenlängen berechnen muss.

Den Namen Radio2Xray habe ich gewählt, weil mein Programm einen Grossteil des Spektrums der elektromagnetischen Wellen berechnen kann: Von Wellen im tiefen Herzbereich (z.B. Radio, Sprechfunk) bis hin zum Exa- und Zettahertzbereich (z.B. Röntgenstrahlung).

1.3 Die wichtigsten Bauelemente von Radio2Xray

Das Programm Radio2Xray ist in mehrere Klassen aufgeteilt, die klar voneinander getrennte Aufgaben erfüllen. Dadurch bleibt der Code übersichtlich, wiederverwendbar und leicht erweiterbar.

1.3.1 Program.cs – Hauptsteuerung des Programms

Die Datei Program.cs enthält die Main-Methode und steuert den gesamten Programmablauf. Sie besteht aus folgenden Hauptelementen.

1.3.1.1 do-while-Schleife

Diese Schleife sorgt dafür, dass das Programm wiederholt ausgeführt wird, bis der Benutzer das Wort „Ende“ eingibt. Damit kann der Benutzer mehrere Frequenzen nacheinander umrechnen, ohne das Programm zwischendurch wieder neu starten zu müssen.

1.3.1.2 Eingabeprüfung

Die Frequenzeingabe wird mit double.TryParse() geprüft, damit nur gültige Zahlen akzeptiert werden. Zusätzlich werden keine negativen Werte angenommen und die Eingabe darf nicht länger als 9 Zeichen sein, um schwer lesbare Werte zu vermeiden.

1.3.1.3 Dateiausgabe

Jede Berechnung wird automatisch in der Datei „Berechnungen.txt“ gespeichert.

1.3.1.4 Benutzerführung

Abstände (Leerzeilen) auf der Konsole dienen der besseren Übersichtlichkeit. Da im Code oft eine oder mehrere Leerzeilen generiert werden, habe ich dazu die Hilfsmethode Utils.Leerzeilen() angelegt.

1.3.2 Physics.cs – Physikalische Berechnungen

Diese Klasse enthält alle Methoden zur Berechnung und Umrechnung der physikalischen Werte. Ich habe meine Methoden modular aufgebaut, damit sie einfacher zu testen sind und in zukünftigen Versionen des Programms auch Bausteinartig für andere Programmelemente wiederverwendet werden können.

1.3.2.1 FrequenzZuHertz(double frequenz, string einheit)

Wandelt Frequenzwerte in beliebigen Einheiten (Hz, kHz, MHz, GHz, …) nach Hertz um.

Beispiel: 1 MHz → 1 000 000 Hz.

1.3.2.2 Wellenlaenge(double frequenz, string einheit)

Berechnet die Wellenlänge mit Werten in SI-Einheit (Frequenz in Hertz, Lichtgeschwindigkeit in m/s und Wellenlänge in Metern) und ruft anschliessend die Methode WellenlaengeHumanreadable() auf.

1.3.2.3 WellenlaengeHumanreadable(double wellenlaengeMeter, string einheitFrequenz)

Gibt die Wellenlänge automatisch in der passenden Masseinheit aus:

Beispiele: 2.99 m, 254 nm (Nanometer) oder 15 cm, je nach Einheit der Frequenz.

1.3.3 Menu.cs – Interaktives Auswahlmenü

Dieses interaktive Menü erlaubt es dem Benutzer, die Frequenzeinheit mit den Pfeiltasten auszuwählen.

1.3.3.1 AuswahlMenu(string[] optionen, string anweisung)

Zeigt alle verfügbaren Einheiten (Hz, kHz, MHz, …), die über den Parameter «string[] optionen» der Methode übergeben werden mit einem Rahmen und farbiger Markierung an.

Mit den Pfeiltasten ↑ und ↓ kann navigiert werden, Enter bestätigt die Auswahl. Die Schleife dieses Menüs reagiert auf eben diesen Tastendruck.

Mit diesem Menü entsteht eine einfache, aber sehr benutzerfreundliche Steuerung. Das Auswahlmenü ist bewusst so gebaut, dass es wiederverwendbar ist. Die Methode kann mit einem beliebigen Array von Strings und einem String, der als Anweisungstext ausgegeben werden soll, als Parameter aufgerufen werden. Falls in Zukunft noch weitere Menüs zu diesem Programm hinzukommen sollen, ist dieses Menü innert Sekunden implementiert.

1.3.4 Utils.cs – Hilfsmethoden

Die Hilfsklasse Utils enthält aktuell erst eine Methode. Weitere Hilfsmethoden werden bei einer Weiterentwicklung meines Programms womöglich dazukommen.

Leerzeilen(int anzahlLeerzeilen)

Gibt eine bestimmte Anzahl Leerzeilen in der Konsole aus. Diese Methode verbessert die Lesbarkeit der Ausgabe und vermeidet doppelte bzw. mehrfache Console.WriteLine()-Aufrufe.

2 Typische Programmdurchläufe

2.1 Nutzer macht eine gültige Eingabe

Der Nutzer wählt zuerst die Einheit der Frequenz, die er eingeben möchte. Das hier sichtbare Menü kann der Nutzer mit den Pfeiltasten nach unten und nach oben navigieren. Mit der Entertaste bestätigt er seine Auswahl.

```
C:\Users\vetschjo\OneDrive -

=========== Wellenlänge berechnen ===========
Dieses Programm berechnet die Wellenlänge anhand deiner eingegebenen Frequenz.

Wähle zunächst die Masseinheit deiner Frequenz.
--> Navigiere mit den Pfeiltasten hoch und runter, bestätige mit der Enter-Taste.

| > Hz |
|  kHz |
|  MHz |
|  GHz |
|  THz |
|  PHz |
|  EHz |
|  ZHz |
```

Für unser Beispiel hier wählt der Nutzer «GHz» als Einheit aus und bestätigt mit Enter.

```
C:\Users\vetschjo\OneDrive -

=========== Wellenlänge berechnen ===========
Dieses Programm berechnet die Wellenlänge anhand deiner eingegebenen Frequenz.


Die Wellenlänge (l) wird berechnet mit
der Lichtgeschwindigkeit (c = 299'792'458 m/s)
und der Frequenz (f):

     c
l = ---
     f

Gib die Frequenz in GHz ein:
```

Der Nutzer macht nun die Eingabe «2.4», weil er die Wellenlänge passend zu 2.4 GHz erfahren möchte. Der Nutzer bestätigt dann mit Enter.

```
C:\Users\vetschjo\OneDrive -

=========== Wellenlänge berechnen ===========
Dieses Programm berechnet die Wellenlänge anhand deiner eingegebenen Frequenz.


Die Wellenlänge (l) wird berechnet mit
der Lichtgeschwindigkeit (c = 299'792'458 m/s)
und der Frequenz (f):

     c
l = ---
     f

Gib die Frequenz in GHz ein: 2.4
Wellenlänge = 12.49 cm

Deine Berechnung wurde in der Datei 'Berechnungen.txt' gespeichert.


Um noch eine Freuquenz in Wellenlänge umzurechnen, drücke die Enter-Taste.
Mit der Eingabe 'Ende' kann das Programm beendet werden.
```

Das Programm berechnet die die Wellenlänge als 12.49cm und speichert diese Berechnung im Text-File.

Der Nutzer kann nun mit der Enter-Taste eine neue Berechnung starten (führt direkt wieder zum interaktiven Menü zur Auswahl der Einheit der Frequenz) oder mit der Eingabe «Ende» kann er das Programm sauber beenden.

2.2 Nutzer macht eine ungültige Eingabe

Bei einer ungültigen Eingabe, in diesem Beispiel «lkaösdkj ölajewij asdölfj klcxyjvxcyövkj dfkj» meldet das Programm dem Nutzer, das als Frequenz nur positive Zahlen eingeben werden dürfen und wenn es eine Kommazahl ist, sollte diese mit '.' als Kommaseparator eingegeben werden.

```
C:\Users\vetschjo\OneDrive -

=========== Wellenlänge berechnen ===========
Dieses Programm berechnet die Wellenlänge anhand deiner eingegebenen Frequenz.


Die Wellenlänge (l) wird berechnet mit
der Lichtgeschwindigkeit (c = 299'792'458 m/s)
und der Frequenz (f):

     c
l = ---
     f

Gib die Frequenz in Hz ein: lkaösdkj ölajewij asdölfj klcxyjvxcyövkj dfkj
Diese Eingabe konnte nicht verarbeitet werden.
Die Frequenz muss eine positive Zahl sein. Kommazahlen bitte mit '.' schreiben.
Gib die Frequenz in Hz ein:
```

Danach ist das Programm direkt wieder bereit, um die Eingabe erneut entgegenzunehmen.

Das Programm lehnt alle Eingaben ab, die keine Zahl sind, kleiner als 0 sind oder länger als 9 Zeichen lange sind. Ist die Zahl der Frequenz tatsächlich länger als 9 Ziffern, so muss der Benutzer eine andere Einheit wählen.

3 Test und Code-Review

Das Programm wurde von Mitschüler Adrian Herbst am 20.10.2025 getestet und erhielt folgendes Feedback:

- Sehr schönes Interface!
- Eingaben werden gut geprüft: Das Programm reagiert robust auf falsche Eingaben (z. B. Buchstaben, negative Zahlen).
- Bei sehr kleinen Frequenzen (z. B. 0.0000000000001 Hz) wurden die Wellenlängen unschön dargestellt.
    - [Anmerkung von Jonas Vetsch:] → Lösung: Ich habe das so verbessert, dass Eingaben nun nicht länger als 9 Zeichen sein dürfen, sonst werden sie abgelehnt.
- Einige Kommentare sind überflüssig.
    - [Anmerkung von Jonas Vetsch:] → Lösung: Ich habe die Code-Kommentare überarbeitet und vereinfacht. Es sind nur noch wirklich relevante Kommentare vorhanden.
- Die Ergebnisse habe ich mit verschiedenen Frequenzeinheiten überprüft und sind physikalisch korrekt.

4 Reflexion

4.1 Zielsetzung und Planung

Ziel war es, ein übersichtliches und interaktives Konsolenprogramm zu entwickeln, das sowohl mathematisch und physikalisch korrekt als auch benutzerfreundlich ist. In der Planungsphase habe ich zuerst die benötigten Klassen (Physics, Menu, Utils) definiert und deren Zuständigkeiten festgelegt.

4.2 Umsetzung

Die Umsetzung erfolgte schrittweise: Zuerst habe ich die physikalische Berechnung in Physics.cs implementiert. Darauf folgte die Erstellung des interaktiven Menüs mit Pfeiltastensteuerung in Menu.cs. Anschliessend integrierte ich in Program.cs die Eingabeprüfung sowie die Dateiausgabe. Ich testete das Programm zwischendurch immer wieder selbst, um Fehler zu finden. Zum Schluss führte ich ein Code-Review mit meinem Sitznachbar Adrian Herbst durch und setzte danach noch die von ihm vorgeschlagenen Verbesserungen um.

4.3 Schwierigkeiten und Lösungen

Sehr kleine oder sehr grosse Frequenzen führten zunächst zu unübersichtlichen Darstellungen. Dem begegnete ich durch eine intelligente, einheitsabhängige Ausgabe der Wellenlänge mit der Methode «WellenlaengeHumanreadable(double wellenlaengeMeter, string einheitFrequenz)» und der Beschränkung der Eingabelänge.

Die Darstellung des Auswahlmenüs war anfangs unübersichtlich, weshalb ich sie mit ASCII-Rahmen, Farbakzenten und Pfeiltasten-Navigation deutlich verbesserte.

Wiederholte Leerzeilen in der Ausgabe machten den Code unnötig lang; hier sorgte eine eigene Hilfsmethode Utils.Leerzeilen() für mehr Übersicht.

4.4 Lernprozess

Ich habe gelernt, wie ich mehrere Dateien und Klassen innerhalb eines C#-Projekts anlege und miteinander verknüpfe. Zudem habe ich gelernt, wie ich ein interaktives Menü bauen kann. Auch habe ich mehr Erfahrungen mit Validierung von Benutzereingaben sammeln können.

Besonders lehrreich war für mich auch die Anwendung der Clean-Code Prinzipien aus dem smartlearn-Modul. Ich hatte intuitiv nicht alles umgesetzt und konnte damit meine Variablennamen und Kommentare verbessern.

4.5 Selbstreflexion und nächste Schritte

Ich bin mit meinem Ergebnis sehr zufrieden, insbesondere, dass es mir gelungen ist die das ganze Frequenzspektrum in meinem Programm abzubilden und dass ich eine fehlerresistente Eingabelogik implementieren konnte.

Für ein nächstes Projekt plane ich, eine grafische Benutzeroberfläche zu ergänzen und optional weitere physikalische Umrechnungen (z. B. Energie oder zusätzliche Frequenzbereiche) zu integrieren.

Insgesamt hat mir das Projekt gezeigt, wie sich mit überschaubarem Aufwand ein nützliches Tool entwickeln lässt, das Physik und Programmierung überzeugend verbindet.